

Build Your First Accessible WordPress Block

macOS · Terminal · LocalWP · VS Code

Equalize Digital · WP Accessibility Meetup

Page 1 / 3

Before You Start — Mac

- ✓ macOS 11 Big Sur or later
- ✓ Node.js v18+ (type: node -v to check)
- ✓ LocalWP running — green dot visible
- ✓ VS Code or any code editor open
- ✓ Terminal app open

i Verify Node.js

Open Terminal and type: node -v
Expected: v20.11.0 (or any v18+)
Download: nodejs.org/en/download

The 4 Files You Will Edit

- edit.js** Controls what the editor sees in the dashboard
- save.js** Generates the HTML saved to DB + shown to visitors
- style.scss** Shared styles: editor + frontend (notice colours)
- block.json** Defines attributes, supports, and metadata

1 Open Terminal in your plugins folder

In LocalWP → right-click site → Open Site Shell.
Or type in Terminal:

```
cd ~/Local\ Sites/my-site/app/public/wp-content/pl...
```

Replace 'my-site' with your actual LocalWP site folder name.

2 Scaffold the block plugin (~60 sec)

Downloads and creates a full block template.

```
npx @wordpress/create-block my-notice-block
```

Type y + Enter if prompted to install the package.

3 Move into the block folder

Run ALL commands from within this folder.

```
cd my-notice-block
```

4 Install dependencies (run once)

This downloads packages into node_modules.

```
npm install
```

Wait 30–90 s until the prompt returns.

5 Start the development compiler

Always use `npm run start` — not just `npm start`.
The extra keyword triggers the full WP build pipeline.

Terminal Commands (Mac)

<code>npm install</code>	Install all packages once after scaffolding
<code>npm run start</code>	Start compiler — watches & rebuilds on save
<code>npm run build</code>	Minified production build for deployment
<code>npm run lint:js</code>	Check JS for errors and style issues
<code>npm run lint:js -- --fix</code>	Auto-fix lint errors where possible
<code>npm cache clean --force</code>	Clear cache if npm run start hangs or fails
<code>ctrl + c</code>	Stop the compiler in the terminal window

6 Activate the plugin in WordPress

WP Admin → Plugins → 'My Notice Block' → Activate.

Not visible? Check the folder name matches step 2.

7 Open all 4 files in VS Code

File → Open Folder → select my-notice-block.

Source files live in: `src/`

`edit.js` · `save.js` · `style.scss` · `block.json`

8 Add noticeType attribute (block.json)

Open `block.json`. Under "attributes" add:

```
"noticeType": {
  "type": "string",
  "default": "info"
}
```

This stores which type the editor selected.

9 Add a type selector (edit.js)

Import `SelectControl`, add inside `InspectorControls`:

```
import { SelectControl } from '@wordpress/componen...
// Inside Edit(), bind to attributes.noticeType:
```

Compiler rebuilds automatically on every file save.

10 Add type as CSS class (save.js)

Read `noticeType` and add it to the wrapper `className`:

```
<section className={
  `notice notice-${attributes.noticeType}`
}>
```

Matches the `.notice--info / warning / error / success` in `scss`.

11 Style in style.scss (already done!)

The uploaded `style.scss` already has all 4 variants.

Each has `bg`, `border-color`, `text`, `icon` from the `scss` tokens.

Check contrast with webaim.org/resources/contrastchecker

12 Insert and test the block

Pages → Add New → type /notice → pick your block.

Right sidebar → change Notice Type.

Save Draft → Preview → Preview in new tab.

Common Errors & Fixes

✗ Red block in editor	Remove, re-add block and retype text — resets saved data
✗ Port 8887 in use	Stop the other LocalWP site first, then retry
✗ node not found	Install Node.js from nodejs.org/en/download
✗ Changes not showing	Normal refresh Cmd+R — do not hard-refresh
✗ npm run start crashes	Delete <code>node_modules</code> , run <code>npm install</code> , retry
✗ Block not in inserter	Check WP Admin → Plugins — is it activated?

Block Handbook

developer.wordpress.org/block-editor

Contrast Checker

webaim.org/resources/contrastchecker

axe DevTools

deque.com/axe/devtools

Node.js

nodejs.org/en/download

LocalWP

localwp.com

Build Your First Accessible WordPress Block

Windows · PowerShell required · Git Bash will cause errors

Equalize Digital · WP Accessibility Meetup

Page 2 / 3

Before You Start — Windows

- ✓ Windows 10 (64-bit) or Windows 11
- ✓ Node.js v18+ LTS — nodejs.org/en/download
- ✓ LocalWP installed — green dot visible
- ✓ VS Code — code.visualstudio.com
- ✓ PowerShell open — NOT Git Bash, NOT CMD

■ Use PowerShell — NOT Git Bash

WordPress build scripts use cross-env and Node path separators that break inside Git Bash on Windows. Open PowerShell: search 'PowerShell' in the Start menu. VS Code: click the terminal dropdown  and choose PowerShell. LocalWP: right-click your site → Open Site Shell (auto-correct).

i Verify Node.js

Open PowerShell and type: `node -v`
Expected: `v20.11.0` (or any v18+)
Download LTS from: nodejs.org/en/download

The 4 Files You Will Edit

-  **edit.js** Controls what the editor sees in the dashboard
-  **save.js** Generates the HTML saved to DB + shown to visitors
-  **style.scss** Shared styles: editor + frontend (notice colours)
-  **block.json** Defines attributes, supports, and metadata

1 Open PowerShell in your plugins folder

LocalWP → right-click site → Open Site Shell.
Or open PowerShell and type:

```
cd 'C:\Users\YourName\Local Sites\
my-site\app\public\wp-content\plugins'
```

Replace *YourName* and *my-site* with your actual names.

2 Scaffold the block plugin (~60 sec)

Downloads and creates a full block template.

```
npx @wordpress/create-block my-notice-block
```

Type *y* + Enter if prompted to install the package.

3 Move into the block folder

Stay here for ALL subsequent commands.

```
cd my-notice-block
```

PowerShell Commands (Windows)

<code>npm install</code>	Install all packages — run once after scaffolding
<code>npm run start</code>	Start compiler — watches & rebuilds on save
<code>npm run build</code>	Minified production build for deployment
<code>npm run lint:js</code>	Check JS for errors and style violations
<code>npm run lint:js -- --fix</code>	Auto-fix lint errors where possible
<code>npm cache clean --force</code>	Clear cache if npm run start hangs or fails
<code>ctrl + c</code>	Stop the compiler in current PowerShell window

6 Activate the plugin in WordPress

Browser → WP Admin → Plugins → 'My Notice Block' → Activate.

Refresh if you do not see it immediately.

Still missing? Verify the path you used in step 1.

7 Open all 4 files in VS Code

File → Open Folder → select `my-notice-block`.

Source files live in: `src/`

`edit.js` · `save.js` · `style.scss` · `block.json`

8 Add noticeType attribute (block.json)

Open `block.json`. Under "attributes" add:

```
"noticeType": {
  "type": "string",
  "default": "info"
}
```

This stores which type the editor selected.

9 Add a type selector (edit.js)

Import `SelectControl`, add inside `InspectorControls`.

```
import { SelectControl } from '@wordpress/componen...
// Bind to attributes.noticeType inside Edit()
```

Compiler rebuilds automatically on every file save.

10 Add type as CSS class (save.js)

Read `noticeType` and add it to the wrapper `className`:

```
<section className={
  `notice notice-${attributes.noticeType}`
}>
```

Matches `.notice-info` / `warning` / `error` / `success` in `scss`.

11 Style in style.scss (already done!)

The uploaded `style.scss` has all 4 variants with WCAG-verified colours (4.5:1 contrast ratios). Check with webaim.org/resources/contrastchecker

12 Insert and test the block

Pages → Add New → type /notice → pick your block.

Right sidebar → change Notice Type.

Save Draft → Preview → Preview in new tab.

Common Errors & Fixes (Windows)

✗ Red block in editor	Remove, re-add block and retype — resets saved data
✗ Port 8887 in use	Stop the other LocalWP site first, then retry
✗ node not found	Install Node.js LTS from nodejs.org/en/download
✗ Changes not showing	Press F5 — do NOT use Ctrl+Shift+R hard refresh
✗ npm run start crashes	Delete <code>node_modules</code> , run <code>npm install</code> , retry
✗ Git Bash path errors	Switch to PowerShell — cross-env breaks in Git Bash
✗ Execution policy error	Run: <code>Set-ExecutionPolicy RemoteSigned -Scope CurrentUser</code>

Block Handbook

developer.wordpress.org/block-editor

Contrast Checker

webaim.org/resources/contrastchecker

axe DevTools

deque.com/axe/devtools

Node.js

nodejs.org/en/download

LocalWP

localwp.com

Build Your First Accessible WordPress Block

Expand Your Block · Attributes · PHP · API · Accessibility

Equalize Digital · WP Accessibility Meetup

Page 3 / 3

A — Adding More Attributes (block.json)

i How attributes work

Every editable value lives in block.json → 'attributes'.
edit.js reads + writes them via setAttributes().
save.js reads them and renders them as static HTML.
Each attribute is saved per block instance in the DB.

A1 Add a heading field to block.json

Under "attributes", add alongside noticeType:

```
"heading": {
  "type": "string",
  "default": ""
}
```

Creates a new data field — empty by default.

A2 Wire it in edit.js with RichText

Import RichText from @wordpress/block-editor:

```
<RichText tagName='h4'
  value={attributes.heading}
  onChange={v => setAttributes({heading: v})}
  placeholder='Optional heading...' />
```

RichText supports bold and italic within the field.

A3 Output it in save.js (only when set)

RichText.Content (note the .Content) renders saved HTML:

```
{attributes.heading &&
  <RichText.Content tagName='h4'
    value={attributes.heading}
  />
}
```

B — Dynamic Rendering with PHP (render.php)

i When to use PHP rendering instead of save.js

Use when the block must show LIVE data on every page load:
latest posts, stock prices, user-specific content.
Add to block.json: "render": "file:./render.php"
Set save() to return null — pick ONE approach only.

B1 Create render.php in the block root folder

WordPress passes \$attributes and \$content automatically.

```
<?php
$t = esc_attr($attributes['noticeType'] ?? 'info');
echo '<div class="notice notice--'. $t. '">'
  . wp_kses_post($content)
  . '</div>';
```

\$attributes matches every key defined in block.json.

C — Connecting to an External API

■ Never put API keys in JavaScript

JS runs in the browser — keys are visible to anyone.
Always proxy API calls through a PHP REST endpoint.

C1 Register a REST endpoint in PHP (plugin root file)

```
add_action('rest_api_init', function() {
  register_rest_route('myblock/v1', '/data', [
    'callback' => 'myblock_fetch_data',
    'permission_callback' => '__return_true'
  ]);
});
```

Endpoint available at: /wp-json/myblock/v1/data

C continued — Fetch the endpoint in edit.js

C2 Call your endpoint from edit.js

Add at the top of edit.js:

```
import apiFetch from '@wordpress/api-fetch';
import { useState, useEffect }
  from '@wordpress/element';

const [data, setData] = useState(null);
useEffect(() => {
  apiFetch({ path: '/myblock/v1/data' })
    .then(setData);
}, []);
```

Show a spinner or placeholder while data === null.

D — Reading WordPress Data (posts, terms, users)

D1 useSelect — read any WP data store in the editor

Import useSelect from @wordpress/data:

```
import { useSelect } from '@wordpress/data';

const posts = useSelect(select =>
  select('core').getEntityRecords(
    'postType', 'post', { per_page: 5 }
  )
);
```

Returns undefined while loading, then an array of objects.

E — Accessibility Must-Haves (from style.scss + save.js)

useBlockProps() Spread {...useBlockProps()} on the outer div in edit.js. Injects required WP ARIA

role='region' Already in save.js — turns <section> into a named, navigable landmark for screen readers.

aria-label Set to noticeType + 'notice'. Without it, role=region is ignored by most screen readers.

aria-live='polite' In save.js — if block content changes after load, screen readers announce it

aria-hidden on icon Icon span has aria-hidden='true'. The type is communicated by aria-label —

Contrast 4.5:1 All scss tokens are WCAG AA verified. Check any new colours at

Not colour alone Each type has icon + label + border + bg. Colour is never the only indicator — WCAG

forced-colors @media (forced-colors: active) in style.scss keeps the border visible in Windows High

F — Feature Ideas to Explore Next

Inner Blocks Use InnerBlocks to let editors drop other blocks

Block Variations Register Info/Warning/Error/Success as

Block Patterns Save a pre-styled notice as a one-click

Block Transforms Convert a Paragraph into a Notice Block

theme.json Expose colours as style attributes to match the

WP_Query CPT Pull site-wide alerts from a custom post type

devtools

Node.js
nodejs.org/en/download

LocalWP
localwp.com